

Online Games Popularity Prediction

Milestone 1 Technical Report

Team ID: CHP_2

Team Members

Name	ID
Menna Ali Thabet	2022170844
Asmaa Waleed Sayed	20231700208
Yousef Samir Atia	20231700263
Ali Mohamed Gaber	20231700234
Abdullah Hany Ahmed	20231700231
Esraa Mahmoud Khakifa	20231700207

Prepared for the project: Online Games Popularity Prediction

Date: 26 April 2026

1. Project Overview

This milestone focuses on predicting the popularity of Steam games using structured metadata, text signals, image metadata, and community-generated information. The target variable is RecommendationCount, and the main goal of this phase was to understand which signals are most related to game popularity and which preprocessing steps were necessary before regression modeling.

The raw dataset contained 11,357 games and 88 columns. After cleaning, duplicate removal, feature construction, and dropping columns that were not useful for modeling, the working dataframe became 11,315 rows and 84 columns. After one-hot encoding and other transformations, the feature space expanded to 144 columns before feature selection.

2. Dataset Analysis

The notebook first examined the structure of the data and the target distribution. The target RecommendationCount was highly skewed, which means the problem is dominated by a small number of very popular games and a large number of low-popularity games. This makes simple linear assumptions weak and motivates careful preprocessing and model selection.

Feature	Spearman r	Interpretation
SteamSpyPlayersEstimate	0.79	Strongest relationship with popularity
SteamSpyOwners	0.75	Very strong positive signal
SteamSpyOwnersVariance	0.75	Ownership-related signal remains useful
Metacritic	0.47	Moderate positive association
AchievementCount	0.33	Useful but weaker than audience-size features
PriceFinal	0.26	Price matters, but less than player-estimate signals

These correlations show that popularity is not driven by a single feature. The strongest evidence comes from audience-size proxies such as SteamSpyPlayersEstimate and SteamSpyOwners. In comparison, quality proxies such as Metacritic and AchievementCount are positive but weaker.

The visual analysis in the notebook also showed several important patterns: more movies/trailers and screenshots tended to increase median recommendations, free games typically attracted more recommendations than paid games, price had a non-linear effect, and older games accumulated more recommendations over time. Release year therefore needed to be treated carefully as a time-related signal rather than a simple standalone feature.

A URL/domain analysis was also performed. SupportURL and Website values were parsed because many developers had no dedicated web presence, while others used recognizable domains such as social pages or official portals. This led to the Support_Tier and Website_Authority features, which capture professionalism and publication style in a more compact way than raw URLs.

3. Preprocessing and Feature Engineering

The preprocessing pipeline was extensive because the dataset contained a mix of booleans, numeric values, text, URLs, list-like strings, dates, and sparse support/requirements fields. The main principle was to convert noisy raw metadata into compact signals while keeping leakage under control.

Technique	How it was implemented	Why it helped
Blank strings -> missing values	Replaced whitespace-only strings with NaN before imputation.	Allowed consistent missing-data handling.
Row inconsistency cleanup	Used more reliable source columns where duplicates or mismatches existed.	Reduced noise from conflicting fields.
Release date parsing	Parsed ReleaseDate and created Release_Year, Release_Month, month sine/cosine, and game age.	Captured seasonality and age effects.
Unified description text	Merged DetailedDescrip, AboutText, and ShortDescrip into Game_Description.	Reduced fragmentation across text fields.
Sparse text flags	Created Has_* indicators for fields like SupportEmail and DRMNotice.	Kept presence/absence signal without sparse noise.
List-like cleanup	Normalized SupportedLanguages and PopularUserTags into tuples.	Made tag counts and language-based features possible.
Text normalization	Removed trademark markers and cleaned special symbols in ordinary text columns.	Standardized text before downstream processing.
Review sentiment	Used a DistilBERT sentiment model on Reviews to create Review_Sentiment.	Converted long review text into a compact polarity feature.
Missing-value imputation	Used IterativeImputer with BayesianRidge for numeric fields; mode fill for categorical fields.	Preserved information while filling gaps.
Outlier handling	Examined boxplots, applied log1p to skewed numeric columns, then winsorized remaining extreme values.	Reduced the impact of heavy-tailed variables.
Encoding	Applied ordinal encoding to Header_Size_Group and one-hot encoding to categorical fields.	Prepared features for regression models.
Scaling	Standardized the final numeric feature set after feature selection.	Helped linear and distance-sensitive models.

Feature engineering also added several higher-level signals: Has_discount and discount_amount, platform_count, category_count, genre_count, language_count_log, header/background image size features, and semantic description embeddings. The description embeddings were produced with all-MiniLM-L6-v2 and reduced with PCA to 30 components, capturing 0.427 of the variance in the notebook.

3.1 Dataset sizes and split strategy

The final modeling split used `train_test_split` with an 80/20 split on indices only, so the dataframe itself remained available during preprocessing. The resulting split sizes were 9,052 training rows and 2,263 test rows.

Set	Size
Training	9,052
Testing	2,263
Validation	No separate validation set; 5-fold CV was used inside GridSearchCV on the training set

3.2 Feature selection and discarded columns

To avoid modeling with the full 144-feature matrix, a three-stage feature selection pipeline was applied. First, `VarianceThreshold` with a threshold of 0.05 was used to remove near-constant features that carry almost no signal regardless of the target. Second, Mutual Information regression scoring was computed on the remaining features to rank them by their statistical dependency with `RecommendationCount`, providing a model-agnostic view of feature relevance. Third, a `RandomForestRegressor` was fitted on the training split to estimate feature importance scores. Features were ranked by cumulative importance and retained until the cumulative sum reached 95% of the total importance mass, keeping 43 features.

The strongest retained features were `SteamSpyPlayersEstimate`, `SteamSpyPlayersVariance`, `tag_count`, `pc_min_word_count`, `AchievementCount`, `PriceFinal`, several semantic description components, header/background image sizes, `SteamSpyOwners`, `SteamSpyOwnersVariance`, and `Release_Year`. After this step, all selected features were numeric, so the final scaling step applied to all of them.

Kept / engineered feature groups	Discarded raw inputs
Game popularity proxies, semantic embeddings, image metadata, tags, price/platform/category richness, language count, and requirement-derived features	<code>QueryID</code> , <code>ResponseID</code> , <code>QueryName</code> , <code>SupportURL</code> , <code>Background</code> , <code>HeaderImage</code> , <code>Website</code> , <code>AboutText</code> , <code>DetailedDescrip</code> , <code>ShortDescrip</code> , <code>Reviews</code> , <code>ReleaseDate</code> variants, <code>ResponseName</code> , <code>SupportEmail</code> , <code>LegalNotice</code> , raw PC/Mac/Linux requirement text, constant columns, duplicates, and very high-missing columns

4. Regression Models

The notebook compared at least two regression families, but in practice it tested several models:

`LinearRegression`, `Ridge`, `Lasso`, `DecisionTreeRegressor`, `RandomForestRegressor`, `GradientBoostingRegressor`, `ExtraTreesRegressor`, and a separate `XGBoost` regressor.

Hyperparameters were tuned with `GridSearchCV` using 5-fold `KFold` cross-validation on the training data.

The main difference between the models is how they learn relationships. Linear models assume an approximately linear relation and are easy to interpret. Ridge adds L2 regularization to reduce coefficient instability. Lasso adds L1 regularization and can shrink weak features toward zero. Tree models capture non-linear patterns and interactions: a single decision tree is interpretable but can

overfit, while Random Forest, Extra Trees, and Gradient Boosting reduce error by combining many trees in different ways.

Model	Train MSE	Test MSE	Train R ²	Test R ²
Linear Regression	3.1703	3.1479	0.6942	0.6987
Ridge	3.4168	3.4408	0.6705	0.6707
Lasso	5.3497	5.2933	0.4840	0.4934
Decision Tree	2.2789	2.4584	0.7802	0.7647
Random Forest	1.0333	2.1077	0.9003	0.7983
Gradient Boosting	0.9800	1.9685	0.9055	0.8116
Extra Trees	1.1390	2.0411	0.8901	0.8046
XGBoost	1.4856	2.0257	0.8567	0.8061

Gradient Boosting achieved the best test R² in the notebook (0.8116) with the lowest test MSE among the compared models (1.9685). XGBoost was extremely close, with a test R² of 0.8061. Extra Trees (0.8046) and Random Forest (0.7983) also performed strongly. Lasso performed the worst of the tested models (Test R² = 0.4934), which suggests that the data is not well explained by a heavily sparse linear fit. Notably, Linear Regression outperformed Ridge in this run (0.6987 vs 0.6707), which indicates that aggressive L2 regularization slightly hurt the linear model on this feature set.

5. Plots and Observations

The figures below summarize the model comparison and the residual behavior of the best model. They were taken from the notebook outputs and included here as evidence of performance.

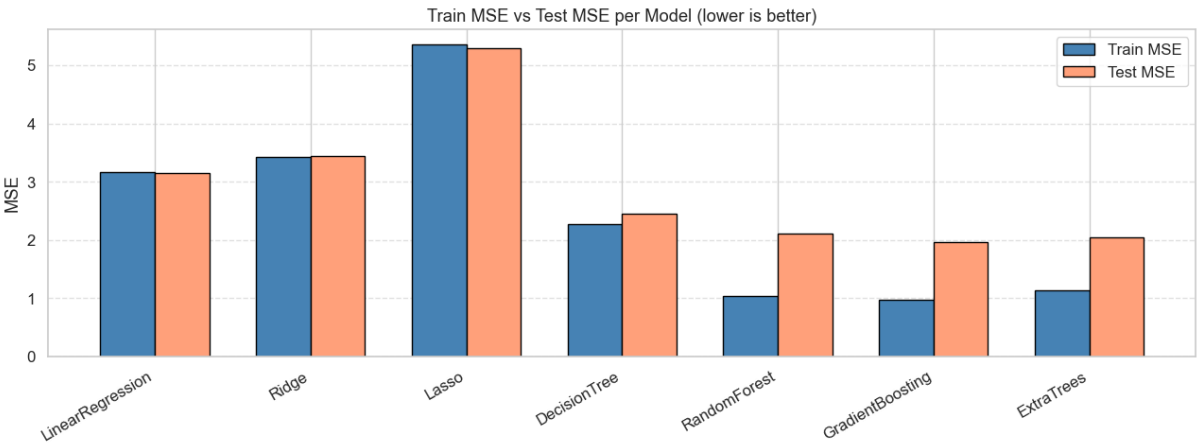


Figure 1. Train and test MSE per model.

Observation: linear models sit around the 3.4 MSE range, while tree ensembles reduce the error substantially. Gradient Boosting and XGBoost are the strongest generalizers among the tested models.



Figure 2. Train and test R^2 per model.

Observation: ensemble trees clearly outperform the linear baselines. Random Forest has the highest train R^2 , but Gradient Boosting gives the best test R^2 , which is the more important result for generalization.

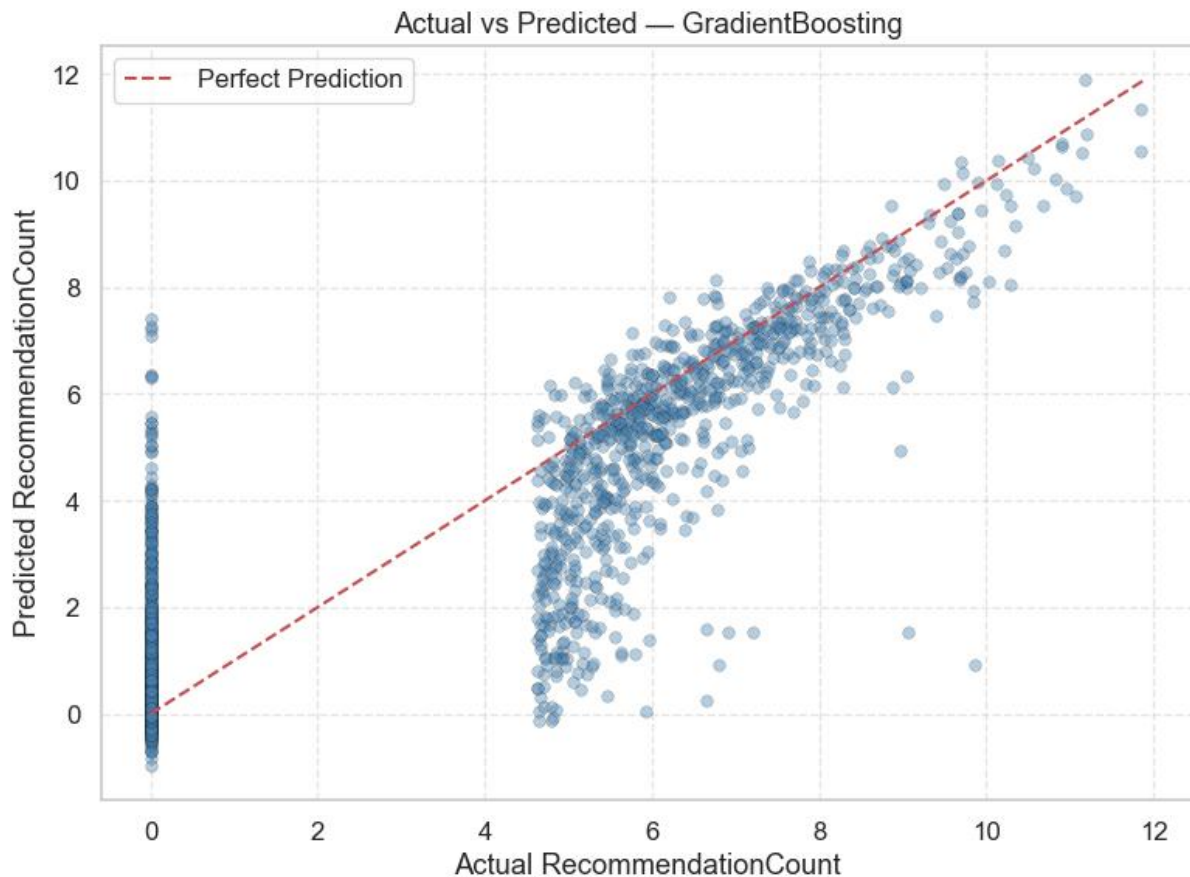


Figure 3. Actual versus predicted values for the best model.

Observation: predictions follow the diagonal trend closely, which means the model captures the overall ordering of popularity well. The spread becomes wider for extreme values, showing that very large recommendation counts are harder to predict precisely.

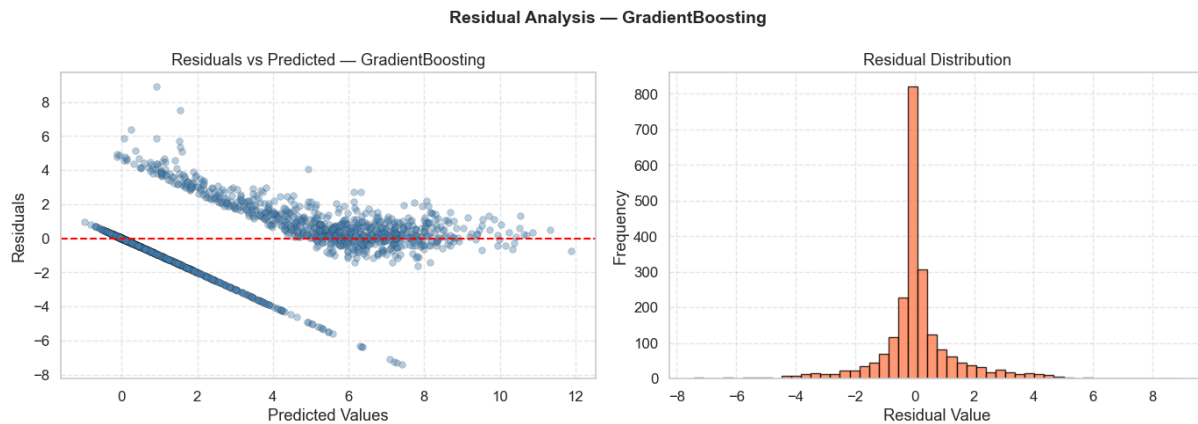


Figure 4. Residual analysis for the best model.

Observation: residuals are centered near zero and the histogram is concentrated around the middle, which is a good sign. The scatter still widens for larger predicted values, indicating that high-popularity games remain the hardest cases.

6. Conclusion

This milestone confirmed the main intuition behind the project: online game popularity is driven mostly by strong audience and visibility signals, not by a single raw field. SteamSpy player and owner estimates were the most important predictors, while price, achievements, media richness, semantic description features, and metadata about developer support and web presence contributed additional value.

The preprocessing work also proved that many raw columns were too noisy or too sparse to use directly. Converting text and list fields into compact engineered features, handling missing values carefully, and reducing the feature space to the most informative variables made the regression problem much easier for the models.

Overall, the linear models provided a useful baseline, but the tree-based ensemble models clearly performed better. Gradient Boosting was the best final choice in this milestone, with XGBoost very close behind. The results support the idea that game popularity depends on a mixture of structured metadata, community signals, and descriptive quality indicators.